DeepStream Pipeline with Kafka, S3, MySQL, and Airflow

Prerequisites

Ensure you have the following dependencies installed:

- **DeepStream 7.0**
- **CUDA 12.x**
- **NVIDIA Drivers**
- **Docker**
- **MySQL**
- **AWS CLI**
- **Kafka**
- **Apache Airflow**

Setup

To initialize the environment, set up Docker services, MySQL database, and AWS S3 bucket by running:

```
sh ./setup.sh
```

⚠️ The **process staging** step takes approximately **50 seconds** to execute.

To gracefully stop Docker services, run:

```
docker compose down
```

PROF

Architecture Overview

1. Data Flow Overview

This system follows a structured pipeline for **real-time video analytics**:

1. **Frame Extraction & Kafka Streaming** - DeepStream detects objects and streams metadata to Kafka.
2. **Kafka to S3 (Staging)** - Messages are collected in batches and stored in Parquet files in an S3 bucket.
3. **S3 to MySQL (Curated)** - Processed Parquet files are loaded into SQL tables for structured storage.
4. **Airflow Orchestration** - Automates processing and ensures synchronization between S3 and MySQL.

2. Data Components

Raw Data (Kafka Messages)

- Kafka topic: `raw`
- Format: JSON

Example Kafka message:

```
{
  "frame_num": 986,
  "object_id": 0,
  "class_id": 0,
  "class_label": "person",
  "confidence": 0.9389,
  "top": 19.5162,
  "left": 2.1036,
  "width": 1358.25,
  "height": 1052.39,
  "sensor_id": "camera-01",
  "mission_id": "mission-01",
  "location_id": "location-01",
  "timestamp": "20250312-235445",
  "latitude": "0.0",
  "longitude": "0.0"
}
```

Staging Data (S3 Parquet Files)

- S3 bucket: `staging`
- Format: Parquet
- Messages are batched (size: **100 messages per file**)
- **Filtering:** Only messages with `confidence > 0.3` are stored

PROF

Curated Data (SQL Tables)

- MySQL database: `curated`
- Structured data stored in relational tables

Database Schema (MySQL)

The MySQL database consists of the following tables:

1. Sensor Table

Stores metadata about the sensors (cameras) generating detections.

```sql
CREATE TABLE sensor (
    sensor_id VARCHAR(255) PRIMARY KEY,
    mission_id VARCHAR(255),
    location_id VARCHAR(255),
    latitude FLOAT,
    longitude FLOAT
);
```

2. Detection Table

Stores detected objects from video frames with confidence levels and bounding boxes.

```sql
CREATE TABLE detection (
    frame_num INT,
    object_id INT,
    class_id INT,
    class_label VARCHAR(255),
    confidence FLOAT,
    top FLOAT,
    `left` FLOAT,
    width FLOAT,
    height FLOAT,
    timestamp DATETIME,
    sensor_id VARCHAR(255),
    PRIMARY KEY (frame_num, object_id),
    FOREIGN KEY (sensor_id) REFERENCES sensor(sensor_id)
);
```

3. Aggregation Table

Stores summary statistics for each sensor, such as total detections and confidence metrics.

PROF

```sql
CREATE TABLE aggregation (
    sensor_id VARCHAR(255) PRIMARY KEY,
    total_detections INT,
    average_confidence FLOAT,
    max_confidence FLOAT,
    min_confidence FLOAT,
    most_frequent_class VARCHAR(255),
    most_frequent_class_count INT,
    last_update DATETIME,
    FOREIGN KEY (sensor_id) REFERENCES sensor(sensor_id)
);
```

4. Processed Files Table

Tracks processed Parquet files to prevent duplication.

```
CREATE TABLE processed_files (
    file_key VARCHAR(255) PRIMARY KEY,
    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Airflow Orchestration (DAGs)

DAG: kafka_to_s3_to_sql_pipeline

This DAG automates the process of detecting new Parquet files in **S3**, processing them, and loading data into **MySQL**.

DAG Tasks:

1. **wait_for_parquet** - Listens for new Parquet files in the `staging` bucket.
2. **curated** - Runs the processing script to load data into MySQL.

DAG Code:

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from datetime import datetime, timedelta

default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2025, 3, 12),
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'kafka_to_s3_to_sql_pipeline',
    default_args=default_args,
    schedule_interval='*/15 * * * *',  # every 15 minutes
    catchup=False,
    description='Orchestrates staging (Kafka to S3) and curated (Parquet
to SQL) steps'
)

wait_for_parquet = S3KeySensor(
    task_id='wait_for_parquet',
    bucket_key='data_*.parquet',  # Match Parquet file patterns
    bucket_name='staging',        # Change to your bucket name if needed
```

```
    wildcard_match=True,
    poke_interval=60,  # Check every minute
    timeout=3600,      # Fail after an hour if no file appears
    dag=dag,
)

curated = BashOperator(
    task_id='curated',
    bash_command='python /opt/airflow/scripts/process_curated.py',
    dag=dag,
)

wait_for_parquet >> curated
```

API Endpoints

1. /raw

Fetches raw data from the Kafka topic.

```
curl -X GET "http://localhost:8000/raw?duration=4"
```

result


```
{"raw_messages":
[{"frame_num":216,"object_id":0,"class_id":0,"class_label":"person","con
fidence":0.9453125,"top":3.392106294631958,"left":0.0,"width":1362.91320
80078125,"height":1064.5472412109375,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234822","latitude":"0.0","longitude":"0.0"},
{"frame_num":217,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94482421875,"top":2.865833044052124,"left":0.0,"width":1362.73
974609375,"height":1064.9205322265625,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234822","latitude":"0.0","longitude":"0.0"},
{"frame_num":218,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94091796875,"top":2.3650896549224854,"left":0.0,"width":1363.4
478759765625,"height":1065.3514404296875,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234822","latitude":"0.0","longitude":"0.0"},
{"frame_num":219,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94384765625,"top":4.28159761428833,"left":0.0,"width":1364.570
068359375,"height":1064.8548583984375,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234823","latitude":"0.0","longitude":"0.0"},
{"frame_num":220,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94677734375,"top":5.832568645477295,"left":0.0,"width":1364.14
```

```
24560546875,"height":1064.094482421875,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234823","latitude":"0.0","longitude":"0.0"},
{"frame_num":221,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94775390625,"top":5.079809188842773,"left":0.0,"width":1363.19
6044921875,"height":1064.263916015625,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234824","latitude":"0.0","longitude":"0.0"},
{"frame_num":222,"object_id":0,"class_id":0,"class_label":"person","conf
idence":0.94677734375,"top":3.7932145595550537,"left":0.0,"width":1363.2
630615234375,"height":1064.782470703125,"sensor_id":"camera-
01","mission_id":"mission-01","location_id":"location-
01","timestamp":"20250312-234824", ......
```


2. /staging

Lists Parquet files in the staging S3 bucket.

```
curl -X GET "http://localhost:8000/staging"
```

result


```
{"staging_files":["data_20250312-233900.parquet","data_20250312-
234756.parquet","data_20250312-234807.parquet","data_20250312-
234817.parquet","data_20250312-234828.parquet","data_20250312-
234840.parquet","data_20250312-234850.parquet","data_20250312-
234900.parquet","data_20250312-234912.parquet","data_20250312-
234922.parquet","data_20250312-234934.parquet","data_20250312-
234944.parquet","data_20250312-234956.parquet"]}%
```


PROF

Retrieve the contents of a specific Parquet file:

```
curl -X GET "http://localhost:8000/staging?
bucket=staging&file=data_20250312-233900.parquet"
```

results


```
{"staging_files":["data_20250312-233900.parquet","data_20250312-
234756.parquet","data_20250312-234807.parquet","data_20250312-
234817.parquet","data_20250312-234828.parquet","data_20250312-
234840.parquet","data_20250312-234850.parquet","data_20250312-
234900.parquet","data_20250312-234912.parquet","data_20250312-
234922.parquet","data_20250312-234934.parquet","data_20250312-
```

```
234944.parquet","data_20250312-234956.parquet","data_20250312-
235006.parquet","data_20250312-235016.parquet","data_20250312-
235026.parquet","data_20250312-235038.parquet","data_20250312-
235048.parquet","data_20250312-235059.parquet","data_20250312-
235110.parquet"]}%
```

3. /curated

Fetches stored detections from MySQL.

```
curl -X GET "http://localhost:8000/curated?table=detection"
```

result

```
......ass_id":0,"class_label":"person","confidence":0.928223,"top":11.98
56,"left":499.539,"width":1098.42,"height":1052.28,"timestamp":"2025-03-
12T23:19:55","sensor_id":"camera-01"},
{"frame_num":5812,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.928711,"top":9.93503,"left":496.719,"width":1097.61,"height"
:1055.58,"timestamp":"2025-03-12T23:19:55","sensor_id":"camera-01"},
{"frame_num":5813,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.878906,"top":3.27233,"left":286.192,"width":1252.29,"height"
:1057.4,"timestamp":"2025-03-12T23:19:56","sensor_id":"camera-01"},
{"frame_num":5814,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.813965,"top":0.0,"left":185.795,"width":1358.5,"height":1059
.77,"timestamp":"2025-03-12T23:19:57","sensor_id":"camera-01"},
{"frame_num":5815,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.788574,"top":0.0,"left":128.498,"width":1425.98,"height":105
8.56,"timestamp":"2025-03-12T23:19:57","sensor_id":"camera-01"},
{"frame_num":5816,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.785156,"top":0.0,"left":228.652,"width":1369.52,"height":106
0.7,"timestamp":"2025-03-12T23:19:57","sensor_id":"camera-01"},
{"frame_num":5817,"object_id":68,"class_id":0,"class_label":"person","co
nfidence":0.926758,"top":18.5336,"left":348.15,"width":1260.85,"height":
1052.78,"timestamp":"2025-03-12T23:19:58","sensor_id":"camera-01"}]}%
```

PROF

Summary

This system enables **real-time** video analytics by processing detections in a scalable, structured manner:

1. **Kafka streams object detections from DeepStream**.
2. **Parquet files store filtered detections in S3**.
3. **MySQL organizes structured detection data**.
4. **Airflow automates processing and keeps data synchronized**.